

AI 使用说明情况（第二次挑战）

HW2 第四部分

1. 使用的 AI 模型

CodeBuddy 智能编码助手（内置大模型，具备沙箱终端、文件读写与联网检索能力）。

2. 使用的提示词（真实摘录）

环节	提示词 / 指令	AI 的实际作用
实验脚本	"帮我写基准脚本：32 条请求、并发 8，共享前缀组 vs 分散前缀组，/generate 提交 input_ids 并流式，要预热、flush_cache、记录 TTFT/TPOT/命中率"	给出 run_benchmark.py 的异步并发骨架与指标计算方式
接口语义	"flush_cache 一直返回 not idle，帮我查服务端实现"	定位到 http_server.py:841：成功固定返回 200 + Cache flushed.，修正判定逻辑
源码主线	"沿 TokenizerManager.generate_request → event_loop_normal → get_new_batch_prefill → match_prefix → worker → cache_finished_req → streaming 帮我定位 v0.5.14 的函数与行号"	给出函数清单与文件:行号，我再逐段精读核对（如 schedule_batch.py:1221 的 set_extend_input_len）
理论与成文	"从因果自注意力公式出发解释前缀复用，并说明 TTFT 与 TPOT 为何变化不同"	协助把公式与机制组织成文，数据仍取自实测结果
可视化	"把主流程画成一张图，标出关键函数所在文件"	生成 SVG 流程图，我再调整布局与配色

3. AI 如何帮助我学习

- **源码导航：**v0.5.14 的调度/缓存模块很大，AI 先给出函数级地图，我再精读关键分支，把"机制—代码—指标"三者对上号。
- **把直觉变成可检验的假设：**"前缀复用只优化 Prefill、不影响 Decode"这一判断，在 AI 协助下变成了两组对照实验，并被 TPOT -1.1% 的数据验证。
- **加速文档与绘图，**让我把时间集中在实验设计与结果核对上。

4. 是否被误导（真实案例）

- **接口语义误判：**AI 依据单次输出认为 /flush_cache 成功时不含括号提示句，导致脚本误判"服务永不空闲"；对照源码后修正。**教训：以源码为准。**
- **负载构造假设错误：**AI 生成的"分散前缀"用循环拼接语料，实际各组前缀相同（分散组命中率 74.9%），加入公共前缀自检后才发现并修复。**教训：负载要断言其统计性质，而不是只看能否跑通。**
- **环境假设偏差：**AI 一开始按 GPU 环境的默认内存参数给命令，而容器限额只有 8 GB，scheduler 被 OOM kill；读 cgroup 指标后改为显式压小 KV 池。**教训：先确认资源约束，再谈参数。**

结论：AI 在"检索、排错、成文"环节帮助很大，但它的经验性假设必须经实测校验。本报告中所有数字均来自脚本真实运行产生的 results/ 原始文件，可逐条回溯。